

Somaiya Vidyavihar University K J
Somaiya School of Engineering

Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 05

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In charge with date

Title: Source Coding

Objectives:

1. To develop a logic and write a program for implementing source coding algorithm for the given problem statement

OUTCOME: apply information theory in source coding

Problem statement: To encode any tongue-twister phrase (given below) using source coding algorithm.

- which wrist watches are swiss wrist watches
- fred fed ted bread and ted fed fred bread
- top chopstick shops stock top chopsticks
- a happy hippo hopped and hiccupped

Procedure:

- Select any one tongue-twister phrase from the given list
- Develop a logic and write a program of source coding algorithms (Huffman coding or LZ coding) to generate a codebook for the symbols in the phrase using MATLAB or any other open source tool.

Expected output:

- Number of different numbers of symbols in the phrase
- Number of code words representing different symbols
- Number of bits needed for coding characters in this phrase using fixed length code
- Theoretical minimum number of bits required to encode the entire phrase assuming that each character is independent of other surrounding characters.
- Number of bits needed for encoding the sequence using codebook derived by Huffman/LZ coding algorithm.
- Calculate efficiency of each codebook.

Code:

```
clc;
clear;
close all;
%% Input phrase
str = 'top chopstick shops stock top chopsticks';
%% Find symbols and frequencies
symbols = unique(str);
freq = zeros(size(symbols));
for i = 1:length(symbols)
    freq(i) = sum(str == symbols(i));
end
N = length(str);
```

```

prob = freq / N;
%% Display frequency table
fprintf('Source Coding using Huffman Algorithm\n\n');
fprintf('Phrase: %s\n', str);
fprintf('Total number of characters = %d\n', N);
fprintf('Number of different symbols = %d\n\n', length(symbols));
fprintf('Symbol\tFrequency\tProbability\n');
for i = 1:length(symbols)
    if symbols(i) == ' '
        fprintf('SPACE\t%d\t%.4f\n', freq(i), prob(i));
    else
        fprintf('%c\t%d\t%.4f\n', symbols(i), freq(i), prob(i));
    end
end
%% Fixed length coding
numSymbols = length(symbols);
fixedBits = ceil(log2(numSymbols));
fixedTotal = N * fixedBits;
%% Entropy
entropy = -sum(prob .* log2(prob));
%% Create Huffman tree
nodes = cell(numSymbols,1);
for i = 1:numSymbols
    nodes{i}.symbol = symbols(i);
    nodes{i}.prob = prob(i);
    nodes{i}.left = [];
    nodes{i}.right = [];
end
while length(nodes) > 1
    % Sort according to probability
    probabilities = cellfun(@(x) x.prob, nodes);
    [~, order] = sort(probabilities);
    nodes = nodes(order);
    % Take two smallest nodes
    leftNode = nodes{1};
    rightNode = nodes{2};
    % Create new parent node
    parent.symbol = [];
    parent.prob = leftNode.prob + rightNode.prob;
    parent.left = leftNode;
    parent.right = rightNode;
    % Remove two nodes
    nodes = nodes(3:end);
    % Add parent node
    nodes{end+1} = parent;
end
root = nodes{1};
%% Generate Huffman codes
codes = cell(numSymbols,1);
codes = generateCodes(root, "", symbols, codes);
%% Display Huffman Codebook

```

```

fprintf('\n Huffman Codebook:\n');
fprintf('Symbol\tFrequency\tCode\n');
for i = 1:numSymbols
    if symbols(i) == ''
        fprintf('SPACE\t%d\t\t%s\n', freq(i), codes{i});
    else
        fprintf('%c\t%d\t\t%s\n', symbols(i), freq(i), codes{i});
    end
end
%% Calculate Huffman encoded bits
huffmanBits = 0;
for i = 1:length(str)
    index = find(symbols == str(i));
    huffmanBits = huffmanBits + length(codes{index});
end
%% Average code length
averageLength = huffmanBits / N;
%% Theoretical minimum bits
theoreticalBits = entropy * N;
%% Efficiency
efficiency = (entropy / averageLength) * 100;
%% Display results
fprintf('\n----- RESULTS ----- \n');
fprintf('Number of different symbols = %d\n', numSymbols);
fprintf('Fixed length code = %d bits/symbol\n', fixedBits);
fprintf('Bits using fixed length coding = %d bits\n', fixedTotal);
fprintf('Entropy = %.4f bits/symbol\n', entropy);
fprintf('Theoretical minimum bits = %.4f bits\n', theoreticalBits);
fprintf('Huffman encoded bits = %d bits\n', huffmanBits);
fprintf('Average Huffman code length = %.4f bits/symbol\n', averageLength);
fprintf('Huffman coding efficiency = %.2f %%\n', efficiency);
fprintf('Bits saved = %d bits\n', fixedTotal - huffmanBits);
%% -----
% Function to generate Huffman codes
% -----
function codes = generateCodes(node, currentCode, symbols, codes)
    % Leaf node
    if isempty(node.left) && isempty(node.right)
        index = find(symbols == node.symbol);
        codes{index} = currentCode;
        return;
    end
    % Left branch = 0
    if ~isempty(node.left)
        codes = generateCodes( ...
            node.left, ...
            [currentCode '0'], ...
            symbols, ...
            codes);
    end
    % Right branch = 1

```